

Vehicle Service Management System

Assignment 2 – Progress
Presentation 1

ITP_IT_46



Modular Responsibilities

Module	Member
Customer Management	Dayananda S.T.P.S.A.
Vehicle Management	Gunawardane C.K.G.
Service Booking + Chatbot	Dharmadasa O.A.P.S.
Service Management	Atthanayake A.M.K.M
Spare Parts & Lubricants Billing & Invoicing	Vidanamage M.S.D. Zinthu N.

An abstract digital cityscape with glowing blue and red cubes and binary code, set against a dark blue background.

SWOT Analysis – Strenghts(Internal Capabilities)

1. Modular Architecture(Design Decision)

- Dividing the MERN stack application into six distinct feature modules
- Develop and test APIs in parallel without causing major code conflicts

2. Data Integrity (Functional Requirement)

- Strict sequential data flow ensures high data integrity
- The system prevents orphaned records and logic errors

3. Cost-Effective Custom Logic (Implementation)

- Implementing custom, heuristic algorithms
- Provides functionality without relying on expensive third-party APIs.

SWOT Analysis - Weaknesses (Internal Limitations)



1. High Module Interdependency:

- system follows a strict sequential flow
- currently requires manual mock data

2. State Management Complexity:

- As individual React components are merged into the single App.jsx shell
- managing the shared state introduces prop-drilling complexities.

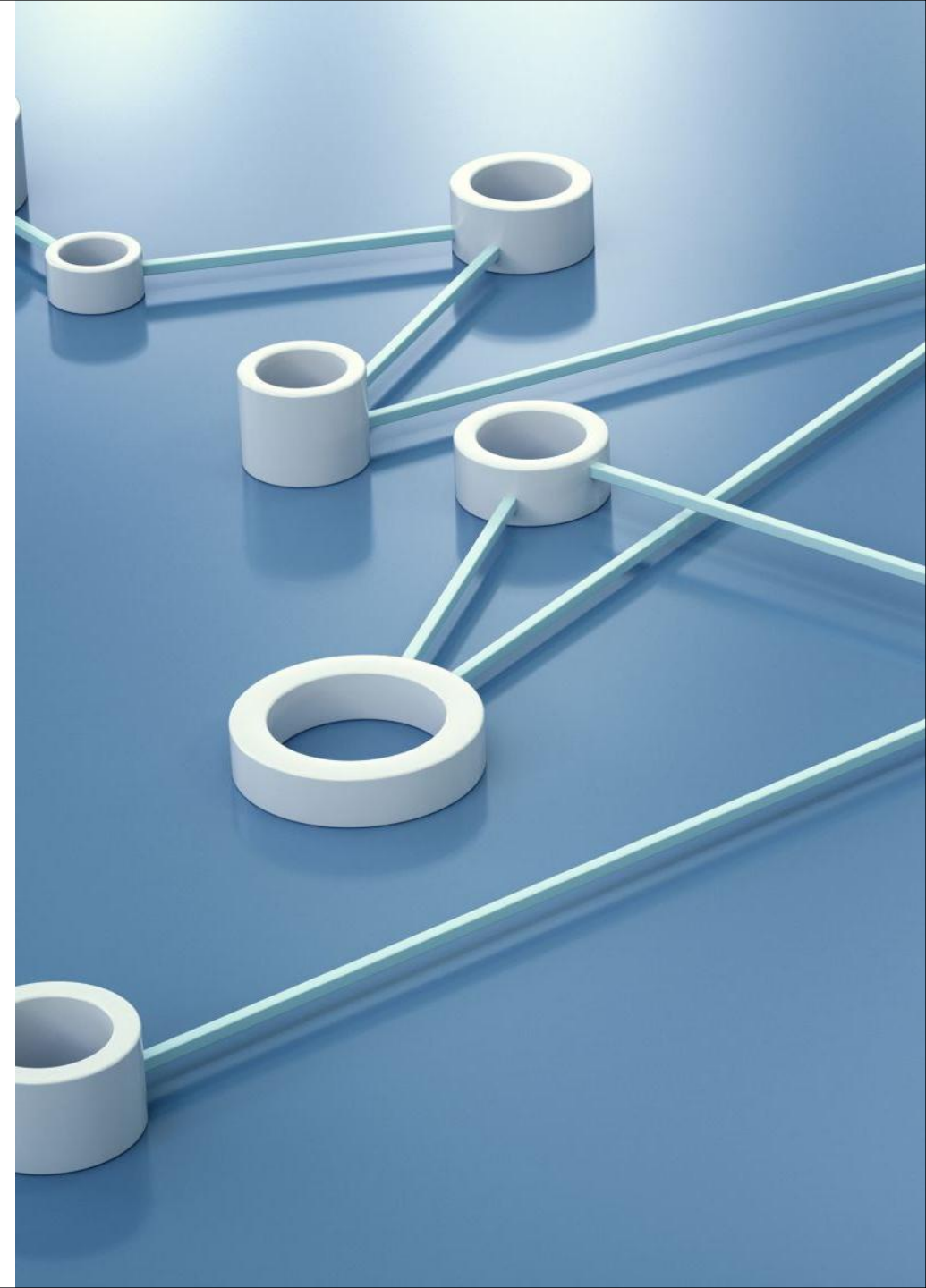
SWOT Analysis - Opportunities (External Possibilities)

1. Database Scalability (Design Decision):

- Choosing MongoDB (NoSQL) over a rigid SQL database provides the flexibility to easily add complex new services
- in the future without requiring massive schema migrations.

2. Future Integrations:

- The modular Express.js backend provides a clear pathway to integrate NodeMailer or SMS gateways later
- allowing the system to send automated booking confirmations or invoice receipts to the customers.



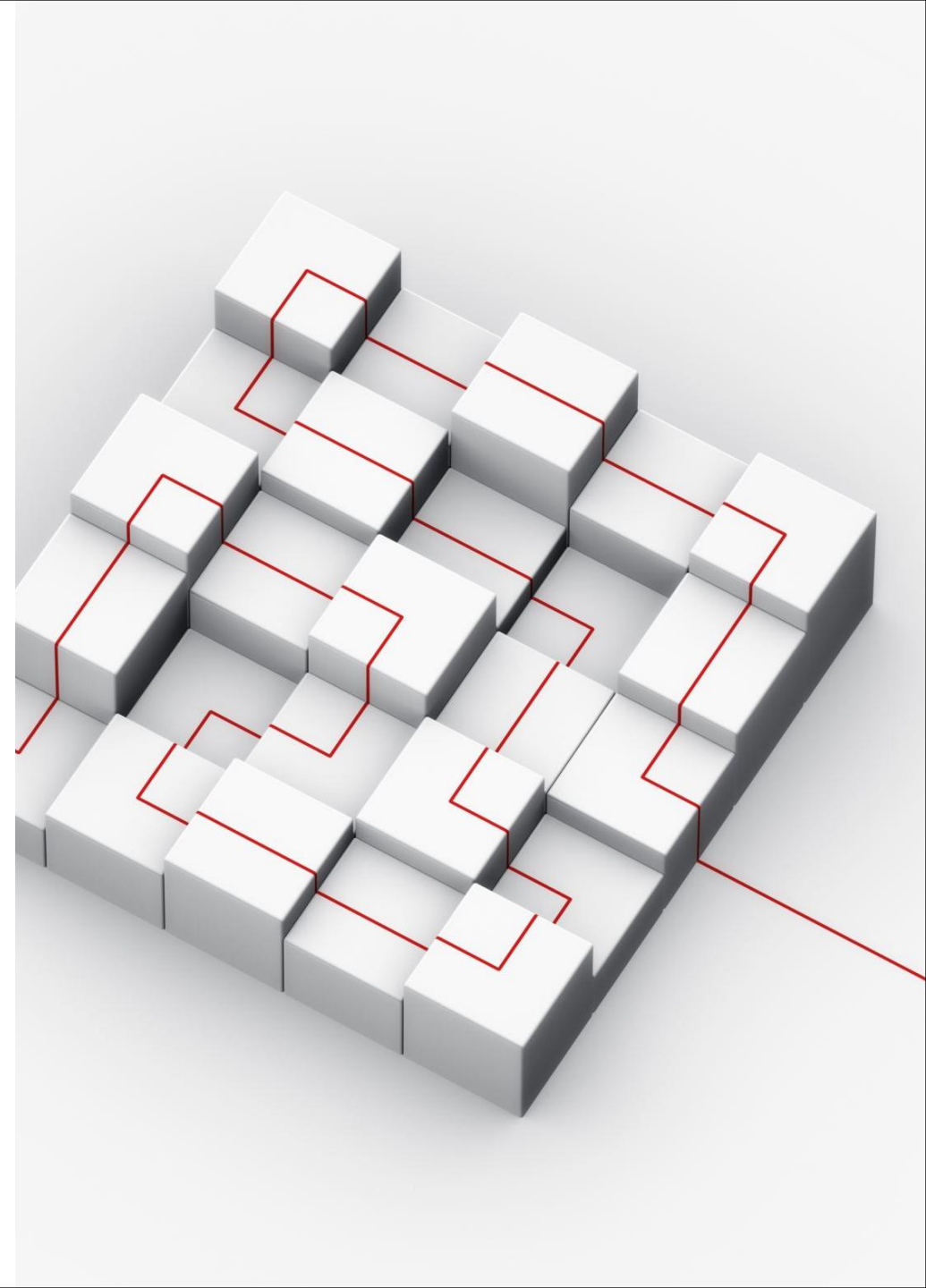
Threats & Risk Management (External Risks)

1. Merge Conflicts During Integration

- *Link to Design:* With six developers pushing code to a single GitHub repository, UI and routing conflicts are highly likely.
- *Handling:* The team implemented a strict "Feature-Folder Architecture" on the frontend and isolated route files on the backend. We also utilize .env files to separate local database URIs, ensuring no developer overwrites another's database.

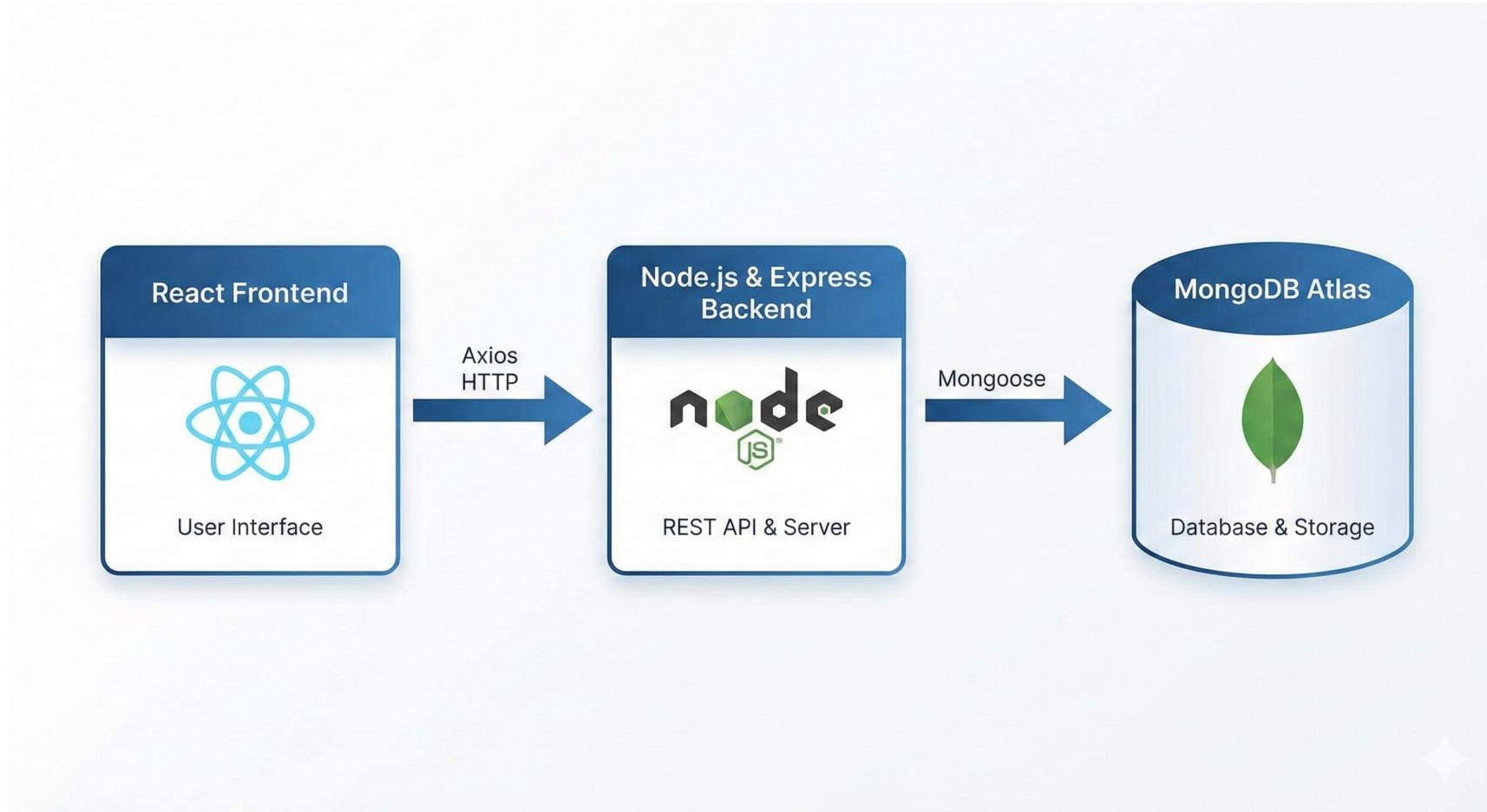
2. Network & CORS Blocks During Presentation

- *Link to Implementation:* Cloud database connections failing due to dynamic university IP addresses, or browser security blocking the frontend from talking to the backend.
- *Handling:* Configured the MongoDB Atlas Network Access to allow 0.0.0.0/0 (Access from anywhere) during the prototype phase, and explicitly implemented the cors middleware in server.js to guarantee seamless API requests during the live demonstration.



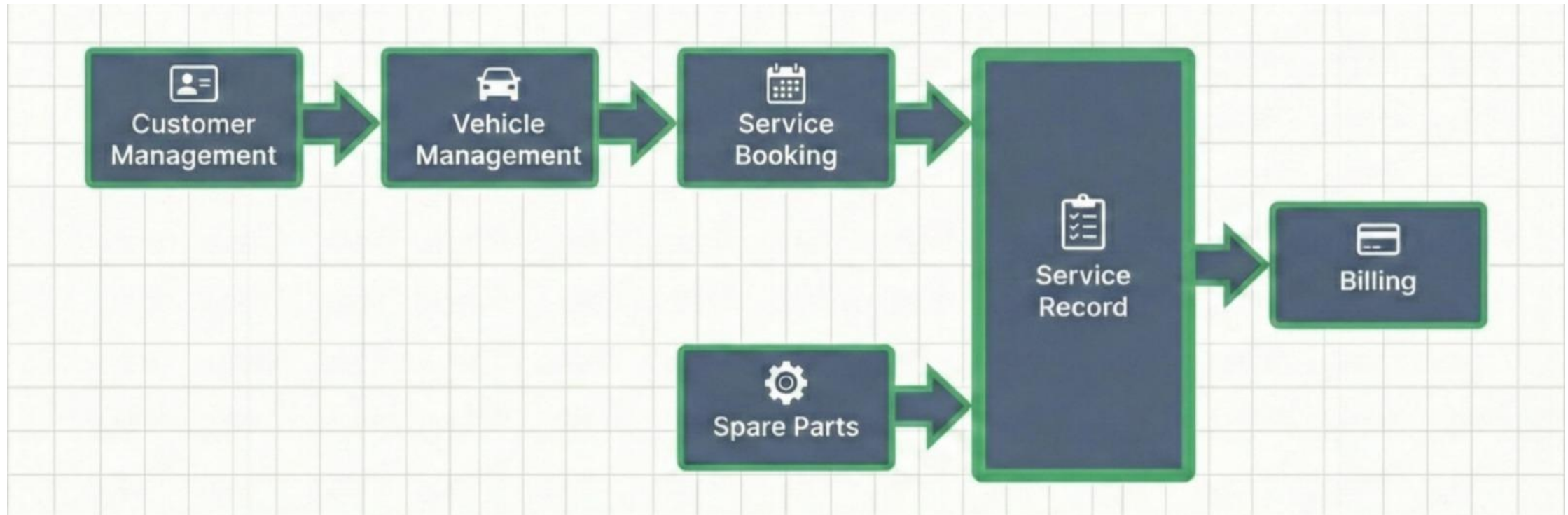
System Architecture & Data Flow

MERN Stack architecture diagram



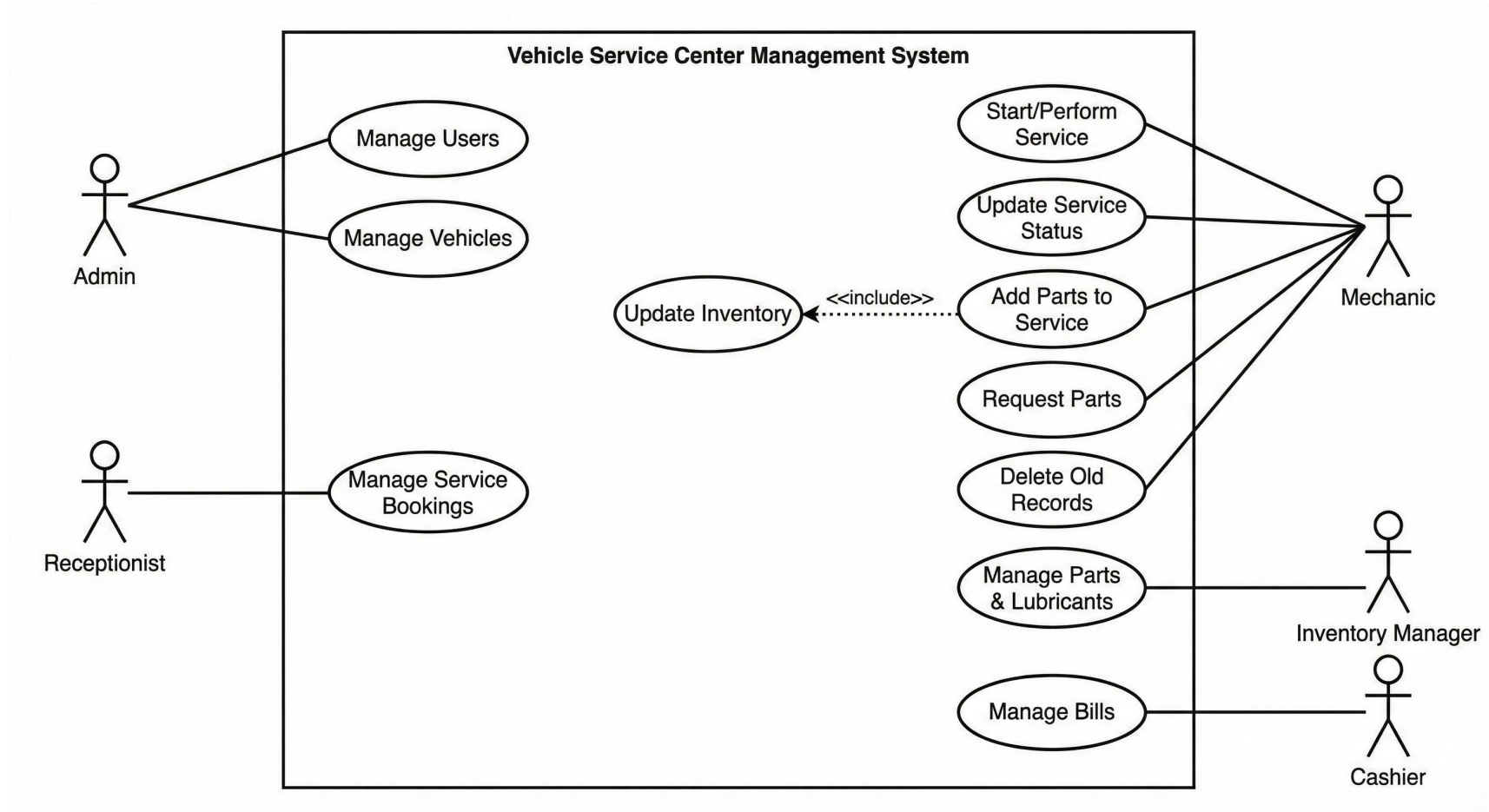
System Architecture & Data Flow

Data Flow Diagram



System Architecture & Data Flow

Use Case Diagram



Customer Record Management

IT24102830

Dayananda S.T.P.S.A

Customer Record Management Data Base

Field Name	Data Type	Key Type	Description / Purpose
_id	ObjectId	System PK	Mandatory MongoDB unique identifier.
customerId	String	Logical PK (Unique)	Readable customer reference (e.g., "CUST-1023").
firstName	String	Normal	Customer's first name.
lastName	String	Normal	Customer's last name.
email	String	Normal	Customer email address, unique for communication.
phoneNumber	String	Normal	Customer contact number.
address	String	Normal	Customer residential address.
registrationDate	String	Normal	Date the customer was registered (YYYY-MM-DD).
status	String	Normal	Default: "Active" (Updates to "Inactive" if account is closed).

Customer Record Management Data Base

- **Custom Identifiers:** Implemented customerId as a readable, logical primary key for easy reference by staff, while retaining _id for backend system relations.
- **Cascading Relationships:** Storing both customerId and vehicleId as foreign keys enforces strict data integrity. This enables dependent dropdown logic in vehicle management, ensuring that operations like service bookings or updates can only be performed for vehicles registered to the selected customer.
- **Optimized Data Access:** Key fields like status and registrationDate are indexed to reduce query time and improve dashboard and reporting performance.
- **Status Management:** The status field allows quick filtering for active/inactive customers, simplifying CRM dashboards and vehicle service workflows.

Customer Record UI Mock-ups

Vehicle Service Center - Customer Record Management

Customer Management

_id6628fdae86f6e49cb72f120e6

Customer ID

First Name

Last Name

Email

Phone Number

Phone Number

Registration Date2024-04-24

Add Customer

Customer List

Customer ID	Name	Email	Phone	Address	Reg. Date	Status	Action
CUST-1023	John Doe	johndoe@email.com	+1 555-123-4567	123 Maple St, Springfield, IL	2024-04-24	Active	EditDelete
CUST-1024	Jane Smith	janesmith@email.com	+1 555-987-6543	456 Oak St, Riverside, CA	2024-04-24	Active	EditDelete
CUST-1025	Mike Johnson	mikejohnson@email.com	+1 555-678-1234	789 Pine St, Austin, TX	2024-04-24	Active	EditDelete

Showing 1 to 3 of 3 entries

Previous

1

Next

Vehicle

Management

IT24104211-

Gunawardhana .C.K.G

Vehical Management

Field Name	Data Type	Key Type	Description
id	ObjectId	System PK	Mandatory MongoDB unique identifier.
vehicleId	String	Logical PK (Unique)	Readable vehicle reference (e.g., "VEH-8055").
customerId	ObjectId	Foreign Key	Links to the Customer Record Management module to identify the owner.
licensePlate	String	Normal	Official vehicle registration number (e.g., "CBA-1234").
make	String	Normal	Vehicle manufacturer (e.g., "Honda").
model	String	Normal	Vehicle model (e.g., "CR-V").
year	String	Normal	Manufacture year (eg., "2019").
mileage	String	Normal	Current odometer reading.
status	String	Normal	Default: "Active" (updates to "Inactive" if the car is sold).

Vehicle Management UI Mock-ups



Vehicle Management Portal

000000

Make:

Toyota

Model:

Corolla

Year:

2022

Mileage:

25000

Status:

Active

Add Vehicle



Vehicle List



All

Toyota Corolla

Plate: CAA-1234

Customer ID: C101

Year: 2018 | Mileage: 45000 km

Status: Active

Edit

Delete

Honda Civic

Plate: CAB-5678

Customer ID: C102

Year: 2020 | Mileage: 30000 km

Status: In Service

Edit

Delete

Service

Booking

IT241000059 -

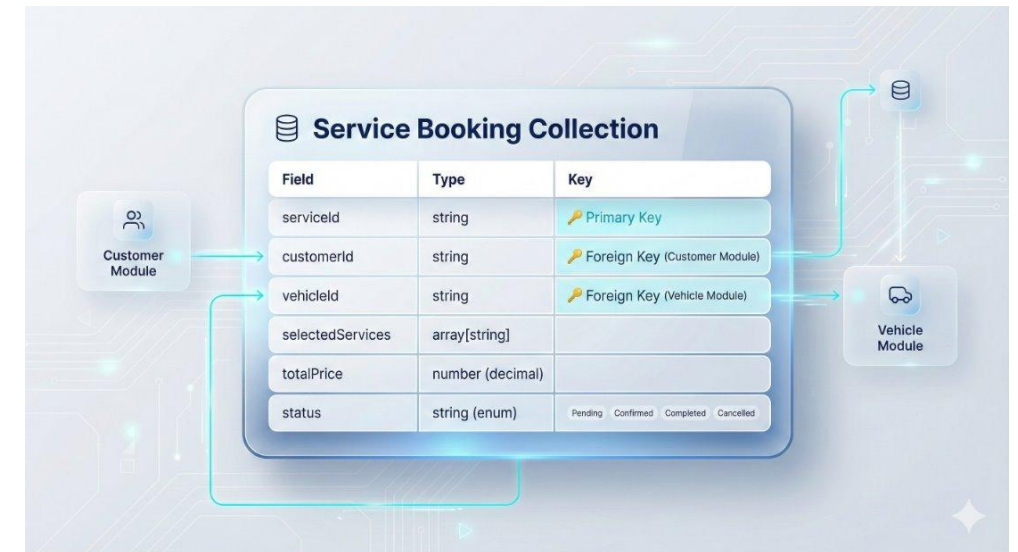
Dharmadasa O.A.P.S.

Service Booking Data Base

Field Name	Data Type	Key Type	Description / Purpose
_id	ObjectId	System PK	Mandatory MongoDB unique identifier.
serviceId	String	Logical PK (Unique)	Readable booking reference (e.g., "BKG-1024").
customerId	ObjectId	Foreign Key	Links to the User Management module.
vehicleId	ObjectId	Foreign Key	Links to the Vehicle Management module.
selectedServices	Array[String]	Normal	Stores multiple services (e.g., ["Wash", "Oil Change"]).
totalPrice	Number	Normal	Pre-calculated estimated cost in LKR.
estimatedTime	String	Normal	Pre-calculated duration (e.g., "45 mins").
date	String	Normal	Service date (YYYY-MM-DD).
timeSlot	String	Normal	Selected time block (e.g., "09:00 AM").
status	String	Normal	Default: "Pending" (Updates to "Completed").

Service Booking Data Base

- **Custom Identifiers:** Implemented `serviceId` as a readable, logical primary key for easy reference by receptionists and customers, while retaining `_id` for backend system relations.
- **Cascading Relationships:** Storing both `customerId` and `vehicleId` as Foreign Keys enforces strict data integrity. It enables our "dependent dropdown" logic, ensuring a booking can only be made for a vehicle that belongs to the selected registered customer.
- **Optimized Data Types:** Pre-calculating `totalPrice` and `estimatedTime` before saving reduces server load during read operations on the dashboard.



Service Booking UI Mock-ups

 **New Booking**

Select Registered Vehicle:

-- Choose Vehicle --

Select Services:

☐	Full Body Wash	 30m 3500 LKR
☐	Interior Cleaning	 45m 4000 LKR
☐	Oil Change	 20m 6000 LKR
☐	Engine Tune-up	 60m 12000 LKR
☐	Brake Inspection	 15m 2500 LKR

 **EST. COST**

0 LKR

 **DURATION**

0 Mins

Date:

mm/dd/yyyy

Start Time:

-- Select Time --

Confirm Booking

Service Booking UI Mock-ups

Upcoming Schedule

mm/dd/yyyy



All Vehicles



No upcoming bookings.

History (Done)

mm/dd/yyyy



All Vehicles



No history records.



Service-Bot



Hi! 🙋 I'm Service-Bot. Ask me about availability (e.g. 'Is 2026-02-15 free?') or check a vehicle (e.g. 'Status of CAB-1234').

Ask Service-Bot...



Service Record Management

IT24101674

Atthanayake A.M.K.M

Service Record Management Data Base

Field Name	Data Type	Key Type	Description / Purpose
_id	ObjectId	System PK	Mandatory MongoDB unique identifier.
recordId	String	Logical PK (Unique)	Readable service record reference (e.g., "REC-5022").
bookingId	ObjectId	Foreign Key	Links to the Service Booking module to pull vehicle and customer info.
usedParts	Array of Objects	Foreign Key	A list of products used. Example: [{ productId: "ObjectId", quantityUsed: 2 }].
mechanicName	String	Normal	The staff member performing the service.
actualDate	String	Normal	Date the service was completed (YYYY-MM-DD).
status	String	Normal	Current status of the service. Default: "In-Progress".

Service Record Management Data Base

- **Logical Service Identifier:** Implemented bookingId as a readable logical reference for staff operations, while retaining MongoDB _id as the system primary key for backend relations.
- **Status Workflow Control:** The serviceStatus field manages lifecycle stages (Pending → In Progress → Completed), ensuring controlled service execution.
- **Cascading Relations:** Each service stores vehicleNumber and customerName to maintain relational consistency with vehicle and customer records.
- **Performance Optimization:** Indexed key fields such as serviceStatus and createdAt to enable faster filtering and dashboard analytics..
- **Timestamp Tracking:** Fields like startedAt and completedAt ensure accurate time-based service tracking and reporting.

Service Record Management

UI Mock-ups

Service Management

Track, update, and complete services quickly.

2 service(s)

Refresh

Add New Service

Create a service record for tracking.

Booking ID

Vehicle Number

Customer Name

e.g., B001

e.g., ABC-1234

e.g., John Doe

Clear

Add Service

Search by booking / vehicle / customer...

All Status

Tip: Use filters to quickly find services.

Booking	Vehicle	Customer	Status	Actions
B003	ABC-1235	John DoERe	Pending	View / Update →
B001	ABC-1234	John Doe	Completed	View / Update →

Tip: Pending → In Progress → Completed. Use "View / Update" to edit notes, cost, status, or delete.

[← Back to list](#)

Service Details

Service Summary

Pending

Service ID: 699d658b0fe48bca45805275

Booking ID: B003

Vehicle: ABC-1235

Customer: John DoERe

Started At: -

Completed At: -

Update Service

Service Status (Admin)

Pending



Labor Cost

0

Service Notes

Start Service

Save

Mark Completed

Delete Service

Spare Parts and Lubrication Management

IT24101356 -
Vidanagamage .M.S.D

Spare Parts & lubrication Data Base

Field Name	Data Type	Key Type	Description / Purpose
_id	ObjectId	System PK	Unique identifier generated by MongoDB.
itemName	String	Normal	Name of the spare part or lubricant (e.g., "Brake Pad", "Engine Oil").
itemType	String	Normal	Type of item: "Spare Part" or "Lubricant".
quantity	Number	Normal	Number of items used or added to stock.
unitPrice	Number	Normal	Price per unit in LKR.
totalPrice	Number	Normal	Calculated cost for this item (quantity × unit price).
dateUsed	String	Normal	Date when the item was used or replaced (YYYY-MM-DD).
status	String	Normal	Default: "Pending". Updates to "Used" or "Replaced" after usage.

Spare Parts & Lubrication UI Mock-ups

Spare Parts & Lubrication Inventory

+ Add New Item

Item Name:

Category:

Brakes

Stock Quantity:

Price (LKR):

Add Item

Inventory

Lubrication

Oil Filter

Category: Lubrication

Price: 1500 LKR

+1

-1

Gear Oil 1L

Category: Lubrication

Price: 1800 LKR

+1

-1

Billing and Invoicing

IT24100348 -
Zinthu.N

Billing and Invoicing Data Base

Field Name	Data Type	Key Type	Description / Purpose
_id	ObjectId	System PK	Mandatory MongoDB unique identifier.
customerName	String	Normal	Saved as plain text for historical accuracy.
vehicleNumber	String	Normal	Vehicle registration number for identification.
billingID	String	Unique	Unique billing number for each invoice.
serviceRecordID	ObjectId	Foreign Key	Link to the service record for the mechanic's data.
sparePartsCost	Number	Normal	Total cost of spare parts used during service.
serviceCost	Number	Normal	Total cost of labor.
totalAmount	Number	Normal	Final total amount to be paid by the customer.
paymentStatus	String	Normal	Shows whether payment is "Pending" or "Paid".
billingDate	String	Normal	Date when the invoice was generated.

Billing & Invoicing UI Mock-ups

 Vehicle Centre – Billing & Invoice

Customer Details

Customer Name

Enter customer name

Phone Number

Enter phone number

Vehicle Plate

e.g. WXY 1234

Services Performed

#	Service	Price (RS)
1	Engine Oil Change	RS 80.00
2	Tyre Rotation	RS 60.00
3	Air Filter	RS 160.00

Subtotal: RS 300.00 | Tax (6%): RS 18.00 | **TOTAL: RS 318.00**

 Print Invoice

ER Diagram

